

CSE 220 — Online 2 Mock Papers (Section C)

Fourier Series & CFT: Coding Problems in Exam Format, Concept-Only Guidance

Assumes your completed offline files `fs_redrawer.py` and `cft_edge_detector.py`

How this document works. These are 24 coding problems written the way the real online specs are written — context, definitions, task bullets, deliverables. No base code is reprinted (use your own offline files), and *no solution code is given*: each answer explains the concept and the algorithm in words, plus the verified expected numbers so you can check yourself. Write every implementation yourself against the clock. Difficulty ramps: Problems 1–4 warm-ups, 5–6 are the actual Section A and B online, 7–24 are Section-C-grade.

Contents

1	Formula Sheet	2
2	Warm-ups	2
3	The Real Onlines	4
4	Section C Territory — Fourier Series Side	6
5	Section C Territory — CFT Side	10
	Final-Hour Checklist	14

1 Formula Sheet

Fourier series (T -periodic, $\omega_0 = 2\pi/T$; the offline calls a_k “ c_n ”):

$$x(t) = \sum_{k=-\infty}^{\infty} a_k e^{jk\omega_0 t}, \quad a_k = \frac{1}{T} \int_T x(t) e^{-jk\omega_0 t} dt, \quad \frac{1}{T} \int_T |x|^2 dt = \sum_k |a_k|^2 \text{ (Parseval).}$$

Time domain	Coefficients	
$\alpha x + \beta y$	$\alpha a_k + \beta b_k$	(same period required)
$x(t - t_0)$	$a_k e^{-jk\omega_0 t_0}$	magnitudes unchanged
$x(-t)$	a_{-k}	index flip
$x(\alpha t)$	a_k unchanged, $\omega_0 \rightarrow \alpha\omega_0$	clock changes, not dict
$x'(t)$	$(jk\omega_0)a_k$	high k amplified
$\int x dt$	$a_k/(jk\omega_0)$, $k \neq 0$	a_0 supplied separately
$x(t)y(t)$	$\sum_p a_p b_{k-p}$	coefficient convolution

Sifting: $\int_L^R x(t)\delta(t-a)dt = x(a)$ iff $L < a < R$, else 0. Impulse train $\leftrightarrow a_k = 1/T$ flat.

Fourier transform: $X(j\omega) = \int x e^{-j\omega t} dt$; $Ta_k = X(jk\omega_0)$. Pairs: rect width $T \leftrightarrow \frac{2\sin(\omega T/2)}{\omega}$ (value T at $\omega = 0$); $\delta(t) \leftrightarrow 1$; $\delta(t - t_0) \leftrightarrow e^{-j\omega t_0}$; shift $\rightarrow e^{-j\omega t_0} X$; $x e^{j\omega_0 t} \rightarrow X(j(\omega - \omega_0))$; $x \cos(\omega_0 t) \rightarrow$ two half copies at $\pm\omega_0$. 2D (offline convention): $F(u, v) = \iint I e^{-j2\pi(ux+vy)} dx dy$; DC at array center (rows//2, cols//2).

2 Warm-ups

Problem 1 — Centroid Extraction and Rigid Translation

Context & Objective. Among all the epicycles of a drawing, exactly one does not rotate. This problem asks you to exploit its geometric meaning.

Task 1: Centroid. In the `FourierEpicycles` class, implement the method `get_centroid`:

- Return the average pen position of the drawing as a single complex number, computed from information already stored in the object. No new integration over the samples is permitted.

Task 2: Translation. Implement the method `translate(dz)`:

- After the call, the reconstructed drawing must be displaced rigidly by the complex offset `dz`, with its shape, size and orientation unchanged.
- Re-integration is prohibited, and `approximate` may not be modified. Identify precisely which stored quantities must change.

Execution & Deliverables. Load any provided SVG; report the centroid value and explain why it comes out as it does for these inputs.

Solution Approach (concepts only — write the code yourself)

$c_0 = \frac{1}{T} \int_0^T f(t) dt$ is by definition the time-average of the pen position — the centroid. It is already sitting in the coefficient dictionary, so the getter is a lookup, nothing more. For translation, note that adding a constant d to a signal changes only its average: by linearity, $f(t) + d$ has the same c_n for every $n \neq 0$ and $c_0 + d$ at zero. So exactly *one* dictionary entry changes. Geometrically, c_0 is the only non-rotating epicycle — the anchor of the chain; moving the anchor moves the whole drawing rigidly. **Expected:** `load_svg_path` centers every shape, so the centroid should print as ≈ 0 (order 10^{-3} or smaller).

Problem 2 — Spectrum Visualization

Context & Objective. Before pruning or filtering a drawing’s harmonics, one must be able to *see* them.

Task: Spectrum Plot. In the `FourierEpicycles` class, implement the method `plot_spectrum`:

- Produce a stem plot of the magnitude of each stored coefficient against its harmonic index n , over the full range $-N \leq n \leq N$.
- Produce a second, aligned stem plot of the phase of each coefficient against n .
- Axes must be labelled with the quantity displayed.

Execution & Deliverables. Run on `svgs/heart.svg` with $N = 150$ and describe the qualitative shape of the magnitude spectrum you observe.

Solution Approach (concepts only — write the code yourself)

Iterate the dictionary in sorted key order to get aligned arrays of n , $|c_n|$ and $\angle c_n$ (magnitude via `abs`, phase via `np.angle`), then two stacked stem plots sharing the x -axis. **Expected:** a handful of tall stems clustered at small $|n|$ (for the heart, $n = -1$ towers over everything) and a tail that decays rapidly — the visual justification for pruning. The phase plot looks scattered; phase carries the *where*, not the *how much* (Problems 7 and 21 make this precise).

Problem 3 — Shape Morphing in the Frequency Domain

Context & Objective. An animation studio wants to morph one drawing smoothly into another. Both drawings are sampled on the same time grid $t \in [0, 2\pi]$, so they share one fundamental frequency.

Definitions. For a blending parameter $\alpha \in [0, 1]$, the intermediate drawing is $z_\alpha(t) = (1 - \alpha) z_{\text{heart}}(t) + \alpha z_{\text{star}}(t)$.

Task: Coefficient-Domain Blending.

- Compute the coefficient dictionaries of `heart.svg` and `star.svg` once each.
- Produce the coefficient dictionary of z_α for each $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ *using only those two dictionaries*. Any further call to `calculate_cn` or `np.trapezoid` during blending is prohibited.
- State the property that guarantees your construction is exact, and the precondition on the two signals it requires.

Execution & Deliverables. Plot the five reconstructed drawings.

Solution Approach (concepts only — write the code yourself)

The property is *linearity*: with the same period (hence the same ω_0 , so the harmonic axes line up), the coefficients of $\alpha x + \beta y$ are $\alpha a_k + \beta b_k$ — because the coefficient integral is linear in the signal. So blend entry-by-entry over the dictionary keys with weights $(1 - \alpha, \alpha)$ and hand the result to a `FourierEpicycles` object for reconstruction. The precondition (same period / same sample grid) is what the spec’s “both sampled on $[0, 2\pi]$ ” guarantees. **Expected:** $\alpha = 0$ and $\alpha = 1$ reproduce the originals exactly; intermediate α give in-between closed curves.

Problem 4 — Signal Energy in Two Domains

Context & Objective. The pruning task of the Section-A online silently relies on one identity: the energy of a periodic signal can be accounted for either in the time domain or coefficient-by-coefficient in the frequency domain.

Definitions. The total energy of the drawing in terms of its Fourier coefficients is $E_{\text{total}} = \sum_{n=-N}^N |c_n|^2$.

Task 1: Implementation. Implement the method `total_energy` in `FourierEpicycles`:

- Return E_{total} computed from the stored coefficients.

Task 2: Numerical Verification.

- Independently compute the time-average power $\frac{1}{T} \int_0^T |f(t)|^2 dt$ from the raw samples using `np.trapezoid`.
- Report both values to six decimal places for `heart.svg`, $N = 150$, and account for any discrepancy.

Solution Approach (concepts only — write the code yourself)

Task 1 is a one-line sum of $|c_n|^2$ over the dictionary values. Task 2 integrates $|f|^2$ over the stored samples and divides by T . They agree by *Parseval's identity*: expanding $\frac{1}{T} \int x \bar{x}$ with the series, every cross term $c_n \bar{c}_m e^{j(n-m)\omega t}$ ($n \neq m$) rotates and averages to zero, leaving $\sum |c_n|^2$. **Expected:** both print 0.635337 for the heart — any residual difference is the energy of harmonics beyond $|n| = 150$, which is negligible here. Remember `abs(c)**2`, not `c**2`: the coefficients are complex.

3 The Real Online

Problem 5 [Section A's actual online] — Energy-Preserving Harmonic Pruning

Context & Objective. In Task 1 of the offline assignment, a 2D drawing is reconstructed using all $2N + 1$ Fourier harmonics. However, for most closed curves, energy is concentrated within a subset of dominant harmonics. Recall that the total energy of a periodic signal in terms of its Fourier coefficients is

$$E_{\text{total}} = \sum_{n=-N}^N |c_n|^2.$$

Your task is to implement energy-based harmonic pruning in `fs_redrawer.py` to compress the shape representation, and evaluate the resulting reconstruction fidelity.

Task 1: Energy-Preserving Harmonic Pruning. In the `FourierEpicycles` class, implement the method `prune_harmonics_by_energy`:

- Given a target energy ratio $r \in (0, 1]$, retain the *minimal* subset of most energetic harmonics such that their cumulative energy accounts for at least a fraction r of the total energy. Discarded harmonics should have their coefficients set to zero.
- The method should return the number of retained non-zero harmonics and the actual energy ratio achieved.

Task 2: Reconstruction Error Evaluation. Implement the method `evaluate_reconstruction_error`:

- Compute and return the Mean Squared Error between the ground-truth samples $f(t)$ and the reconstruction $\hat{f}(t)$: $\text{MSE} = \frac{1}{M} \sum_{i=1}^M |f(t_i) - \hat{f}(t_i)|^2$.

Execution & Deliverables. In the main execution block, load `svgs/heart.svg` with $N = 150$. For target ratios $r \in \{0.96, 0.98, 0.99, 1.00\}$: perform pruning; compute the retained count, actual energy ratio, and MSE; print the results as a console table (Target Ratio | Harmonics Retained | Actual Energy Ratio | MSE); and save the visual comparison plot as `heart-pruned_{r}.png`.

Solution Approach (concepts only — write the code yourself)

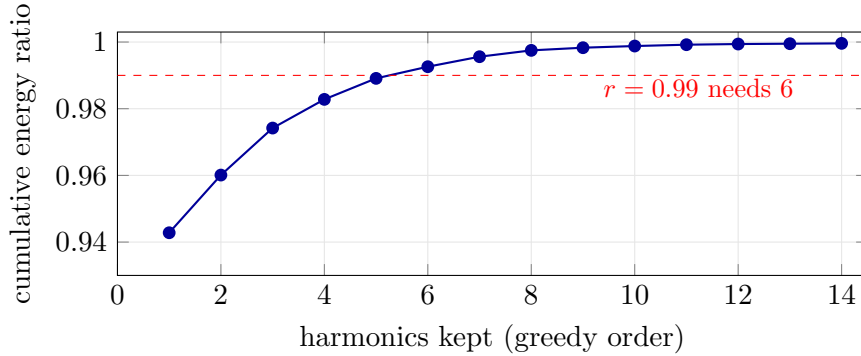
Algorithm. Parseval makes $|c_n|^2$ the energy of harmonic n , so “minimal subset reaching r ” is solved greedily: (1) build the energy of every harmonic; (2) sort harmonic indices by energy, *descending*; (3) walk the sorted list accumulating energy, but **check before taking**: stop the moment

the accumulated energy already meets $r \cdot E_{\text{total}}$ — taking one more would break minimality; (4) zero every coefficient not taken (assign complex zero; do *not* delete keys, or **approximate** and the animation code break); (5) return the kept count and $\text{acc}/E_{\text{total}}$. The MSE method is one reconstruction over `self.t` followed by the mean of squared complex magnitudes — note $|\cdot|^2$ must be `np.abs(...)**2` since the residuals are complex. In the main block, build a *fresh* object per r (pruning is destructive!).

Expected output (verified):

Target r	Retained	Actual ratio	MSE
0.96	2	0.9601	2.545×10^{-2}
0.98	4	0.9828	1.094×10^{-2}
0.99	6	0.9926	4.700×10^{-3}
1.00	301	1.0000	5.782×10^{-8}

Two harmonics carry 96% of the heart ($n = -1$ alone: 94.3%). The greedy pickup order is $-1, 2, -3, -2, -4, 1, \dots$ — energy decides, not index, and it is not $\pm$ -symmetric. At $r = 1.00$ everything survives and MSE is $\sim 10^{-8}$, not zero: the $|n| > 150$ truncation tail remains.



Problem 6 [Section B's actual online] — Band Filters, Complementarity, DC Manipulation

Definitions. For a CFT spectrum of size $H \times W$, the center is $(c_i, c_j) = (\lfloor H/2 \rfloor, \lfloor W/2 \rfloor)$. Let $d(i, j) = \sqrt{(i - c_i)^2 + (j - c_j)^2}$ be the Euclidean distance of entry (i, j) from the center.

Task 1: Band-Pass and Band-Stop Filters. Implement the following filters in the `FrequencyFilter` class. Both apply to the real and imaginary spectrum arrays.

- **Band-Pass:** retain entries where $r_{\text{low}} < d(i, j) \leq r_{\text{high}}$, zeroing the rest.
- **Band-Stop:** zero entries where $r_{\text{low}} < d(i, j) \leq r_{\text{high}}$, retaining the rest.

Task 2: Complementarity Check. Since the two filters above are exact complements, their reconstructed signals satisfy $I_{bp} + I_{bs} \approx I_{recon}$. In `ReconstructionValidator`, check this property on the *raw inverse-CFT arrays* by computing $\delta = \max_{x,y} |I_{bp} + I_{bs} - I_{recon}|$. Return `True` if $\delta < 10^{-9}$, else `False`, along with δ .

Task 3: DC Component Manipulation. The exact center of the spectrum corresponds to the average brightness (DC component) of the image. In `FrequencyFilter`, implement `shift.brightness` by adding `shift.amount` strictly to the *real* component of this center pixel, leaving all other components unchanged.

Solution Approach (concepts only — write the code yourself)

Task 1. Either copy `high_pass`'s double loop with the ring condition $r_{\text{low}} < d \leq r_{\text{high}}$, or (faster and safer) build a boolean ring mask once — distances from a `meshgrid` of row/column indices — and multiply both arrays by the mask (band-pass) or its negation (band-stop). Respect the strict-vs-inclusive inequalities exactly as written, or boundary rings get double-counted/dropped. Work on copies; do not mutate the caller's arrays.

Task 2. One line of algebra is the whole story: the two masks partition the entries, so $F_{bp} + F_{bs} = F$ *exactly*, and the inverse transform is linear in the spectrum, so the reconstructions add the same way. Compute δ with a max of absolute differences. The graded trap: compare against the *round-tripped* I_{recon} , never against the original image — forward+inverse numerical error is far larger than 10^{-9} , but it is *common* to all three arrays and cancels in δ . **Expected:** $\delta \sim 10^{-16}$ – 10^{-15} , comfortably True.

Task 3. Index $[\text{rows}/2, \text{cols}/2]$ of the *real* array, add the amount, touch nothing else (copies again). Why it works: $F(0,0) = \iint I \, dx \, dy$ is the total brightness, and lifting one spectrum sample by A adds a constant $A \cdot \Delta u \Delta v$ (its integration cell) to every reconstructed pixel. **Expected:** for a $[-1,1]^2$ image, $\Delta u = \Delta v = \frac{1}{2}$, so `shift_amount=2.0` brightens every pixel by exactly 0.5 (verified: std of the difference $\sim 10^{-16}$) — hence the entry point’s clip to $[0,1]$.

4 Section C Territory — Fourier Series Side

Problem 7 — Retiming the Animation

Context & Objective. An animator wants the epicycle GIF to begin its trace at a different moment of the drawing cycle, without recomputing anything from the samples.

Task: Time Shift. Implement `shift_in_time(t0)` in `FourierEpicycles`:

- After the call, `approximate(t)` must return $f(t - t_0)$ for every t .
- Only the stored coefficient dictionary may be modified; re-integration and changes to `approximate` are prohibited.

Deliverables. For $t_0 = 0.7$ on `heart.svg`: (a) what happens to the traced *path* over one full period; (b) what changes in the GIF; (c) for each harmonic, which of $|c_n|$, $\angle c_n$ changed, and by how much.

Solution Approach (concepts only — write the code yourself)

The time-shift property: $f(t - t_0) \leftrightarrow c_n e^{-jn\omega t_0}$ — multiply each stored coefficient by that phase factor (an n -dependent rotation: higher harmonics rotate more, because the same delay costs a faster wave more of its cycle). The multiplier has magnitude 1, so: (c) every $\angle c_n$ rotates by $-n\omega t_0$ while every $|c_n|$ is untouched; therefore (a) the set of points traced over a full period is *identical*; and (b) the GIF shows the same circles with the same radii drawing the same outline, merely starting at a different point of it. Distinguish carefully from Problem 1’s `translate` (moves the drawing, touches only c_0) — a favourite trap. **Expected check:** reconstruction after the shift matches an interpolation of $f(t - 0.7)$ to about 2×10^{-3} (truncation only).

Problem 8 — Playing the Drawing Backward

Context & Objective. A reviewer requests the animation traced in the opposite direction, then a combination of reversal with delays — all without touching the samples.

Task 1: Reversal. Implement `reverse_time` in `FourierEpicycles`:

- After the call, `approximate(t)` must return $f(-t)$. Only the coefficient store may change.
- Report the visible effect on the epicycle GIF.

Task 2: Realization of a Composition.

- Using only `shift_in_time` and `reverse_time`, obtain the drawing $f(4 - t)$. Document the exact call sequence, arguments included, and justify the order.

Solution Approach (concepts only — write the code yourself)

Reversal swaps the roles of $+n$ and $-n$: $f(-t) \leftrightarrow c_{-n}$, so rebuild the dictionary with each key's value taken from its negated key. Every epicycle now spins the opposite way; the GIF traces the same outline backward. For $f(4-t)$: write it as $g(-t)$ where $g(t) = f(t+4)$. So first `shift_in_time(-4)` (an *advance*: multiplier $e^{+j4n\omega}$), then `reverse_time()`. After the flip, the phase factors land on the negated indices — exactly the textbook result $c_{-n}e^{-j4n\omega}$. Doing the calls in the other order also has a correct variant, but with `shift_in_time(+4)`; the reliable recipe is substitute inside-out and track what the dictionary currently represents after each call.

Problem 9 — Geometric Editing in the Frequency Domain

Context & Objective. A designer wants standard geometric transformations of a drawing applied directly to its stored frequency representation.

Task: Rotation and Scaling. Implement `transform_drawing(s, theta)` in `FourierEpicycles`:

- After the call, the reconstructed drawing must be scaled by s and rotated by θ about the origin, both rigidly.
- Re-integration is prohibited; `approximate` may not be modified.

Deliverables. State the rule by which each coefficient changes, and what happens to each epicycle's radius, speed, and starting angle. Of the three geometric operations now available (translate, rotate, scale), identify the one that modifies exactly *one* coefficient, with justification.

Solution Approach (concepts only — write the code yourself)

Scaling by s and rotating by θ is multiplication of every *value* of the signal by the single complex constant $se^{j\theta}$. Linearity holds for complex constants, so every coefficient is multiplied by that same constant: each epicycle's radius grows by s , its starting angle advances by θ , its speed $n\omega$ is untouched. The one-coefficient operation is *translation* (only c_0 ; Problem 1) — rotation and scaling touch all coefficients uniformly, and time shift (Problem 7) touches all of them *non-uniformly*.

Expected check: multiplying all coefficients by j must equal $j \cdot \hat{f}(t)$ pointwise — machine precision ($\sim 10^{-16}$).

Problem 10 — Changing the Clock

Context & Objective. The client wants the same drawing traced faster. Time scaling behaves differently from every other property — this problem probes whether you know *where* it acts.

Task: Time Scaling. Implement `rescale_time(alpha)` in `FourierEpicycles`, realizing $f(\alpha t)$:

- After the call, the animation must trace the identical shape, α times faster per unit time.
- The coefficient dictionary must remain byte-for-byte untouched — identify which stored attributes must change instead, and to what values.

Solution Approach (concepts only — write the code yourself)

Substituting αt into the series gives $\sum c_n e^{jn(\alpha\omega)t}$: the same coefficients ride a new frequency ruler. So the method edits the object's *clock*: $\omega \rightarrow \alpha\omega$ and $T \rightarrow T/\alpha$, and deliberately does not touch the dictionary. Everything downstream (`approximate`, the animation timing that reads `fs.omega/fs.T`) then produces the same shape traced $\alpha\times$ faster — all epicycle speeds $n\omega$ scale together, radii and starting angles unchanged. If you find yourself modifying coefficients here, you have answered a different question.

Problem 11 — The Pen's Velocity

Context & Objective. The velocity of the drawing pen, $f'(t)$, is itself a periodic complex signal with its own Fourier series — obtainable without ever numerically differencing the samples.

Task: Derivative Coefficients. Implement `derivative_coeffs` in `FourierEpicycles`:

- Return the coefficient dictionary of $f'(t)$, computed purely from the stored coefficients. Finite differences on the samples are prohibited.

Execution & Deliverables. Verify on `circle.svg`, for which the exact derivative is known in closed form; report the maximum error. Explain the per-harmonic multiplier geometrically (size and direction), and why the derivative's spectrum is dominated by the highest harmonics — and what that implies for drawings with corners.

Solution Approach (concepts only — write the code yourself)

Differentiating each term multiplies its coefficient by $j n \omega$. Geometry of the multiplier: harmonic n is circular motion of radius $|c_n|$ at angular speed $n\omega$; speed is $\omega r = |n\omega||c_n|$ (the $n\omega$ factor), and velocity points *tangent* to the circle — a 90° turn from the position vector, which is exactly what multiplying by j does. Because the multiplier grows linearly in $|n|$, the highest harmonics are amplified the most: a shape whose coefficients decay like $1/n^2$ (anything with corners) has a derivative decaying only like $1/n$ — barely convergent, visually jittery near star tips and cusps; the next derivative would put impulses at the corners (jump $\rightarrow$ impulse rule). **Expected:** the circle is $z = e^{jt}$ with exact derivative je^{jt} ; the coefficient-domain derivative matches it to $\sim 10^{-15}$.

Problem 12 — Integration and the Lost Constant

Context & Objective. Integration should undo differentiation — but one piece of information is irrecoverably destroyed along the way, and your interface must account for it.

Task: Integral Coefficients. Implement `integral_coeffs(mean_value=0.0)` in `FourierEpicycles`:

- Return the coefficient dictionary of $\int f(t) dt$, computed purely from the stored coefficients.
- Every harmonic index present in the input must be present in the output.
- Identify the harmonic whose output value cannot be derived from the input, justify why, and document the role of `mean_value`.

Deliverables. Demonstrate the round trip `derivative_coeffs` $\rightarrow$ `integral_coeffs` and characterize exactly what is and is not recovered. State the condition on c_0 under which the integral of a periodic signal is itself periodic.

Solution Approach (concepts only — write the code yourself)

Divide each coefficient by $j n \omega$ — except $n = 0$, where that would divide by zero, and rightly so: differentiation maps *every* constant to the same derivative, so no integration rule can decide the antiderivative's mean. That is precisely what `mean_value` supplies (compare the sawtooth derivation in the lectures, where a_0 is computed separately as an average). The round trip returns every $n \neq 0$ coefficient exactly (the factors cancel) and sets c_0 to the supplied value — identity minus the mean. Periodicity condition: the running integral gains $c_0 T$ per period, so it is periodic iff $c_0 = 0$ (true for our centered SVGs).

Problem 13 — Products of Signals

Context & Objective. Two drawings sampled on the same period are multiplied pointwise as complex signals. The product's coefficients must be produced without ever forming the

product signal.

Task: Implementation. Implement `multiply_coeffs(other)` in `FourierEpicycles`:

- Return the coefficient dictionary of $f(t)g(t)$, computed *purely* from the two stored dictionaries. Multiplying the sample arrays and re-integrating is prohibited.
- State the harmonic range of the output when both inputs run over $|n| \leq N$, and ensure your data structure accommodates it.

Deliverables. Validate numerically on $x(t) = 1 + \cos(\omega_0 t)$, $p(t) = x^2(t)$, whose exact coefficients are $c_0 = \frac{3}{2}$, $c_{\pm 1} = 1$, $c_{\pm 2} = \frac{1}{4}$.

Solution Approach (concepts only — write the code yourself)

One term from each series multiplies to $a_p b_q e^{j(p+q)\omega t}$ — it contributes to harmonic $k = p + q$. Summing all such pairs gives the *convolution* $c_k = \sum_p a_p b_{k-p}$: the same machine as polynomial multiplication (substitute $z = e^{j\omega_0 t}$) and as LTI convolution. Implement as a double loop over both dictionaries, accumulating into `out[p+q]` — which naturally handles the range trap: the product's harmonics reach $|k| \leq 2N$, wider than either input; a fixed-size structure sized for N silently drops the ends. **Expected:** the validation input has stems $(\frac{1}{2}, 1, \frac{1}{2})$ at $k = -1, 0, 1$; self-convolving yields exactly $(\frac{1}{4}, 1, \frac{3}{2}, 1, \frac{1}{4})$ at $k = -2..2$ (verified numerically to 10^{-4} with sampled coefficients).

Problem 14 — Pruning Under an Error Budget

Context & Objective. A client specifies a hard *error* budget rather than an energy ratio: after compression, the reconstruction MSE must be guaranteed not to exceed ε . Reconstructing repeatedly to check is too slow — the guarantee must come from theory.

Task 1: The Identity.

- State the relationship between the MSE of a pruned reconstruction and the energies of the discarded harmonics, and why it holds.

Task 2: Implementation. Implement `prune_to_mse(eps)` in `FourierEpicycles`:

- Zero out as many harmonics as possible while guaranteeing $\text{MSE} \leq \varepsilon$.
- Any call to `approximate` (or any other reconstruction) is prohibited.

Task 3: The Limit of Pruning.

- Build a 0/1 square-wave signal on $[0, 2\pi]$, reconstruct with $N = 50$, and report the maximum value of the reconstruction.
- Explain why this maximum does not approach 1 as N increases, and what quantity *does* improve with N .

Solution Approach (concepts only — write the code yourself)

Task 1. The error signal after discarding set D is exactly the discarded series $\sum_{n \in D} c_n e^{jn\omega t}$. Apply Parseval *to the error*: its average power — which the MSE over uniform samples approximates — equals $\sum_{n \in D} |c_n|^2$. *MSE = discarded energy*. (Verified on the heart: 4.7005×10^{-3} vs 4.6959×10^{-3} ; the sliver is the $|n| > N$ tail.)

Task 2. This turns Problem 5 inside out: sort *ascending* by energy (weakest first) and discard while the running discarded total stays $\leq \varepsilon$; stop before the discard that would exceed the budget. Same sort as Problem 5, walked from the opposite end, with the same check-before-acting discipline. No reconstruction ever happens — the guarantee is Parseval's.

Task 3. Expected: maximum ≈ 1.0895 — an $\sim 8.95\%$ overshoot beside each jump. This is the *Gibbs phenomenon*: near a discontinuity the partial sum overshoots by a fixed fraction of the jump height, no matter how large N is; growing N only narrows the overshooting spike. MSE improves with N (the spike thins); the max-error does not. Practical echo: pruned drawings wobble precisely at corners and cusps.

Problem 15 — Fixed-Budget Compression Across Shapes

Context & Objective. Storage for the compressed drawing is capped at K coefficients, whatever the energy outcome; separately, the four provided shapes compress very differently, and you must explain why.

Task 1: Implementation. Implement `prune_to_count(K)` in `FourierEpicycles`:

- Keep exactly the K most energetic harmonics, zeroing the rest.
- Return the energy ratio actually achieved.

Task 2: Analysis. Account for this verified table — in particular the circle’s row and the heart’s last column — using a general principle relating a curve’s geometry to its coefficient decay:

shape	$r = 0.96$	$r = 0.99$	$r = 0.999$
circle	1	1	1
star	2	3	4
heart	2	6	11
infinity	4	5	7

Solution Approach (concepts only — write the code yourself)

Task 1 is Problem 5 with the stopping rule “count reached” instead of “energy reached”: sort descending by energy, keep the first K indices, zero the rest, return kept-energy over total. Task 2: a circle traced at constant speed *is* a single complex exponential — one harmonic holds essentially everything, which is why its row is all 1s. The governing principle: **smoothness in time** $\Leftrightarrow$ **decay in frequency**. Smooth curves have rapidly decaying $|c_n|$; corners and cusps (star tips, the heart’s point) push energy into high harmonics with only $\sim 1/n^2$ decay, so banking 99.9% of the energy needs many more of them — the heart’s cusp costs it 11. This is JPEG’s founding observation, per the lecture’s applications slide.

5 Section C Territory — CFT Side

Problem 16 — The Low-Pass Filter and Its Complement

Context & Objective. The offline provided `high_pass`; its opposite number is missing, and the relation between them is itself exploitable.

Task 1: Low-Pass. Implement `low_pass(real, imag, cutoff)` in `FrequencyFilter`:

- Retain entries with $d(i, j) \leq \text{cutoff}$, zeroing the rest. The DC entry must be retained.
- Predict the visual character of its reconstruction before running it.

Task 2: High-Pass Without `high_pass`.

- Produce the high-pass-filtered spectrum without calling `high_pass`, using only your low-pass output and the unfiltered spectrum. Justify with one algebraic identity, and state the corresponding identity between the three *reconstructed images*.

Solution Approach (concepts only — write the code yourself)

Low-pass is the disk mask $d \leq \text{cutoff}$ applied to both arrays; its reconstruction keeps smooth shading and kills edges — a *blurred* image (the lecture’s “blurry-but-clean”). Boundary care: if you build it from a band mask with a lower edge, the lower edge must sit *below* zero so the DC entry ($d = 0$) is included — excluding it silently darkens the result. Task 2: the disk and its exterior partition the entries, so $F_{hp} = F - F_{lp}$, entry by entry — subtract the arrays. By linearity the same identity holds after inversion: $I_{hp} = I_{recon} - I_{lp}$, i.e. “edges = image minus its blur” (the classic unsharp mask). Verified: exact zeros both ways.

Problem 17 — Data-Driven Cutoff Selection

Context & Objective. Choosing the high-pass cutoff radius by hand is guesswork. It should instead be derived from the spectrum’s own energy distribution — the same energy-budget reasoning as the pruning online, transplanted to the frequency plane.

Definitions. The energy of spectrum entry (i, j) is $\Re^2 + \Im^2$; the disk of radius ρ is $D_\rho = \{(i, j) : d(i, j) \leq \rho\}$.

Task: Cutoff From Energy. Implement `cutoff_for_energy(real, imag, r)` in `FrequencyFilter`:

- Return the smallest integer radius ρ such that D_ρ contains at least a fraction r of the total spectral energy.
- A fresh scan of the full array per candidate ρ is not accepted; a single accumulation pass is expected.

Deliverables. State how this problem combines the core algorithms of the two sections’ online.

Solution Approach (concepts only — write the code yourself)

Bucket, accumulate, threshold: compute each entry’s integer ring index $\lfloor d \rfloor$ (meshgrid of index distances, cast to int), sum the energy $\Re^2 + \Im^2$ into one bucket per ring in a single pass (`np.bincount` with weights, or a dictionary of ring sums), take the cumulative sum over increasing radius, normalize by total energy, and return the first radius whose cumulative value reaches r (`np.searchsorted` on the cumulative array). This is Problem 5’s greedy-cumulative-threshold skeleton wearing Problem 6’s radial geometry — one algorithm, two exam costumes. 2D Parseval is what licenses calling $\Re^2 + \Im^2$ “energy”. **Sanity:** on a random test image the disk $d \leq 5$ already held 77% of the energy; a natural photo concentrates far more.

Problem 18 — Diagnosing and Curing the Ripples

Context & Objective. The offline’s edge maps show faint ripples radiating from every strong edge. They are not a bug in your integration — they are a predictable consequence of the filter’s shape, and a differently shaped filter removes them.

Task 1: Diagnosis.

- Explain the origin of the ripples using a transform pair from the lectures, identifying precisely which design choice in `high_pass` produces them.

Task 2: Cure. Implement `gaussian_high_pass(real, imag, sigma)` in `FrequencyFilter`:

- Weight each entry by $H(d) = 1 - e^{-d^2/2\sigma^2}$, d measured from the spectrum center.
- State why this profile eliminates the ripples, and whether Q10-style complementarity survives for the pair $\{H, 1 - H\}$, with justification.

Solution Approach (concepts only — write the code yourself)

Diagnosis: the hard 0/1 disk is a *rectangle* profile in frequency. The $\text{rect} \leftrightarrow \text{sinc}$ pair works in both directions: a sharp box in one domain is an oscillating, slowly-decaying sinc in the other, so masking the spectrum with a hard edge convolves the image with a sinc-like kernel whose side lobes print rings around every strong edge — the 2D sibling of Gibbs (Problem 14: both are sharp truncation in frequency). Cure: use a mask with no edge. The Gaussian weight rises smoothly from 0 at DC toward 1; a Gaussian’s transform is again a Gaussian — no side lobes, no rings. Implementation is the same meshgrid-distance pattern with H multiplying both arrays (weights need not be binary — filters are entrywise weights, full stop). Complementarity survives: $H + (1 - H) = 1$ everywhere, and linearity of the inverse does the rest — the check needs weights summing to one, not membership in $\{0, 1\}$.

Problem 19 — The Rectangle Transform, Numerically

Context & Objective. The centered rectangle's transform $X(j\omega) = \frac{2\sin(\omega T/2)}{\omega}$ (value T at $\omega = 0$) is the most reused pair in the course. You will rebuild it with the offline's integration tool.

Task 1: Numerical Verification.

- Sample the width-2 centered rectangle on $[-5, 5]$ and compute $X(j\omega)$ on $\omega \in [-10, 10]$ using `np.trapezoid` only.
- You may compute a single real integral per frequency — justify, from a symmetry of the signal, why the other component vanishes.
- Report the maximum deviation from the analytic formula.

Task 2: Duality.

- Repeat for width 6 and state the general relationship between a rectangle's width and its spectrum's main-lobe width.

Solution Approach (concepts only — write the code yourself)

Sample the rect, then for each ω integrate `rect` $\times \cos(\omega t)$ with `np.trapezoid` — that is the real part of the transform. The imaginary part is $-\int x \sin(\omega t) dt$, and for an *even* real signal that integrand is odd, so it vanishes by symmetry: one real integral per frequency suffices (the same even/odd bookkeeping as the offline's cosine/sine split). **Expected:** max deviation $\approx 5 \times 10^{-3}$ from $2\sin(\omega)/\omega$ — pure trapezoid discretization. Width 6 gives $2\sin(3\omega)/\omega$: first zero moves from $\omega = \pi$ to $\pi/3$. General law: main-lobe width scales as $1/T$ — short in time is wide in frequency and vice versa; the extreme cases are the one-pixel image (Problem 21) and the constant image.

Problem 20 — Injecting Oscillation into an Image

Context & Objective. Lecture 7's modulation property was stated for 1D signals. Here you will watch it act on a 2D image spectrum, and reason about its interaction with the filters of Problem 6.

Task: Modulation.

- Multiply every row of the image by $\cos(2\pi u_0 x)$ with $u_0 = 20$, using array broadcasting (explicit Python loops prohibited).
- *Before* recomputing the spectrum, predict what `plot_magnitude` will show — where any features appear and their relative strength; then verify.
- State the consequence for a subsequently applied high-pass or band-pass filter.

Solution Approach (concepts only — write the code yourself)

$\cos = \frac{1}{2}(e^+ + e^-)$ splits the spectrum into two half-strength copies: $\frac{1}{2}F(u - u_0, v) + \frac{1}{2}F(u + u_0, v)$. Prediction: the central energy blob vanishes from the center and reappears as *two* blobs at $u = \pm 20$, unchanged in v — displaced horizontally in the plot, because columns are the u axis. Implementation: build the 1D cosine over `img.x` and broadcast it across rows (shape it $(1, W)$); x varies along columns. Filter consequence: content that used to be “low frequency” now sits $u_0/\Delta u$ pixels from the center, so a high-pass *keeps* what it previously killed, and a band-pass ring can be aimed to select exactly the modulated content. Modulation moves signals between the kept and killed zones of every radial filter.

Problem 21 — The One-Pixel Image

Context & Objective. The most extreme image possible — all black except one bright pixel — exposes, in pure form, how position is encoded in a spectrum.

Task 1: Centered Pixel.

- Construct an all-black image with a single bright pixel at the exact center; *predict* the magnitude spectrum and the `imag` array before computing; verify with `compute_cft`.

Task 2: Off-Center Pixel.

- Move the pixel off-center. Classify each of `real`, `imag`, and the magnitude as *changed* or *invariant* relative to Task 1, supporting each classification with a formula and the computed output.

Solution Approach (concepts only — write the code yourself)

The single pixel is a sampled impulse; sifting collapses the transform integral to $F(u, v) = C e^{-j2\pi(ux_0+vy_0)}$ (with C the pixel's small area weight $\Delta x \Delta y$). Centered ($x_0 = y_0 = 0$): $F \equiv C$ — `real` constant, `imag` ≈ 0 , magnitude perfectly flat; `plot_magnitude` shows a featureless field. The narrowest signal needs all frequencies equally — Problem 19's duality at its extreme. Off-center: `real` $= C \cos(2\pi(ux_0 + vy_0))$ and `imag` $= -C \sin(\cdot)$ become striped (changed), but the magnitude $\sqrt{\Re^2 + \Im^2} = C$ stays exactly flat (invariant): position is stored *entirely in phase* — the 2D twin of Problem 7's "delay changes every phase, no magnitude".

Problem 22 — Hidden Redundancy in the Spectrum

Context & Objective. Every spectrum figure in this course looks point-symmetric about its center. That is a theorem about real-valued inputs — worth both a speed-up and a debugging tool.

Task.

- Establish the exact relationship between $F(-u, -v)$ and $F(u, v)$ for a real image, and translate it into statements about the `real` and `imag` arrays under a 180° index flip.
- Verify both statements numerically with array-slicing assertions on a computed spectrum.
- State the runtime optimization this licenses in `compute_cft`, and the self-check it provides.

Solution Approach (concepts only — write the code yourself)

Conjugating the transform integral flips the exponent's sign, which for real I is the same as evaluating at $(-u, -v)$: hence $F(-u, -v) = \overline{F(u, v)}$ — *conjugate symmetry*, the 2D version of $a_{-k} = \overline{a_k}$ for real 1D signals. Because `u/v` are symmetric linspaces, negating a frequency is exactly reversing an index: `real` must equal its `[:, :-1, :, :-1]` flip (symmetric) and `imag` must equal *minus* its flip (antisymmetric). **Expected:** both assertions hold to exactly 0.0. Optimization: compute only half the (v, u) grid and fill the rest by flip-and-negate — nearly 2× on the forward transform (same idea inverts). Self-check: if a fresh implementation breaks either flip identity, hunt for a sign error in the sine term.

Problem 23 — Photometric Editing in the Frequency Domain

Context & Objective. An image's spectrum cleanly separates "average level" from "variation about the average". Editing the two groups independently realizes two classic photo adjustments — if you can predict which is which.

Task 1. Implement `scale_brightness(real, imag, factor)` in `FrequencyFilter`:

- Only the DC entry may change; it is multiplied by `factor`.

Task 2. Implement `boost_contrast(real, imag, gamma)`:

- Every entry *except* the DC entry is multiplied by `gamma`; the DC entry must be exactly preserved.

Deliverables. Predict the visual effect of each; contrast both with Problem 6's

`shift.brightness`; identify the post-processing the entry point must apply and why.

Solution Approach (concepts only — write the code yourself)

DC carries the mean brightness; all other entries carry deviations from the mean. Scaling DC by γ scales the mean level proportionally (a multiplicative brightness change, vs `shift.brightness`'s additive offset). Scaling everything *except* DC by γ scales every deviation while pinning the mean — contrast. Implementation discipline: copy the arrays; for contrast, the cleanest route is scale-everything-then-restore-DC (save the center values first). Linearity guarantees the reconstruction changes by exactly the intended amount. Post-processing: $\gamma > 1$ pushes pixels outside $[0, 1]$, so the entry point must clip. Unifying view: `high_pass` is `boost_contrast` pushed to the extreme — DC and its neighbourhood scaled by zero.

Problem 24 — A Spectrum Split with a Guarantee

Context & Objective. Problem 6 partitioned the spectrum into two complementary pieces. A compression pipeline needs the general version: any number of concentric bands whose sum is *exactly* the original — by construction, not by luck.

Task: Multi-Band Split. Implement `multi_band_split(real, imag, radii)` in `FrequencyFilter`:

- Return `len(radii)+1` spectrum pairs (low / mid / ... / high) whose entrywise sum equals the input spectrum exactly, for *any* radii. Pay attention to boundary entries — including $d = 0$ — so no entry is dropped or double-counted.
- Generalize Problem 6's complementarity check to k bands.
- Report each band's share of the total spectral energy.

Solution Approach (concepts only — write the code yourself)

Build consecutive half-open annuli $(r_0, r_1], (r_1, r_2], \dots$ from the edge list $\{-1\} \cup \text{radii} \cup \{\infty\}$ and reuse the band-pass for each. The half-open convention is the whole guarantee: every entry's distance falls in exactly one interval, so the masks sum to the all-ones array *by construction* — the first edge must sit below zero so $d = 0$ (DC) belongs to the lowest band, and the last must be ∞ so the corners of the array are not orphaned. The k -band check sums all returned `real` arrays (and all `imag`) and asserts equality with the input — or reconstructs each band and checks the images sum to the round-trip, exactly as in Problem 6. Energy shares are per-mask sums of $\Re^2 + \Im^2$ over the total (2D Parseval). Typical decomposition of a photo: low band = blurry base, mid = shapes and soft edges, high = fine texture — JPEG's keep/quantize/drop trio.

Final-Hour Checklist

- **Energy of a harmonic/entry:** `abs(c)**2 /  $\Re^2 + \Im^2$` ; total by Parseval. Sort a dict by value: `sorted(d, key=d.get, reverse=True)`.
- **Greedy threshold pattern** (Problems 5, 14, 17): sort or bucket by energy $\rightarrow$ cumulative sum $\rightarrow$ stop at target; decide check-before vs check-after deliberately ("at least r " = check-then-take).
- **Zero out, never delete** dictionary entries; copy arrays before mutating.
- **Center pixel:** `(rows//2, cols//2)`; distances via `meshgrid`; complements via `~mask`; respect $r_{\text{low}} < d \leq r_{\text{high}}$ boundaries; DC needs the lower edge below zero.
- **Complementarity:** compare with the round-tripped reconstruction, never the original

image.

- **Coefficient edits:** translate $\rightarrow c_0+d$; rotate/scale $\rightarrow \text{all} \times se^{j\theta}$; delay $\rightarrow \text{all} \times e^{-jn\omega t_0}$; reverse $\rightarrow \text{swap } \pm n$; derivative $\rightarrow \times jn\omega$; integral $\rightarrow \div jn\omega$ with c_0 supplied; product $\rightarrow$ convolve dicts (range doubles!); faster clock $\rightarrow$ edit ω/T , not the dict.
- **MSE after pruning = discarded energy** — error questions need no reconstruction.
- Fresh object per experiment — pruning and filtering are destructive.